Multi-Agent Reinforcement Learning for Real-Time Dynamic Production Scheduling in a Robot Assembly Cell

Dazzle Johnson, Gang Chen, and Yuqian Lu

***Abstract*—As industry rapidly shifts towards mass personalisation, the need for a decentralised multi-agent system capable of dynamic flexible job shop scheduling (FJSP) is evident. Traditional heuristic and meta-heuristic scheduling methods cannot achieve satisfactory results and have limited application to static environments. Recent Reinforcement Learning (RL) approaches that consider dynamic FJSP, lack flexibility and autonomy as they use a single-agent centralised model, assuming global observability. As such, we propose a Multi-Agent Reinforcement Learning (MARL) system for scheduling dynamically arriving assembly jobs in a robot assembly cell. We applied a Double DQN-based algorithm and proposed a generalised observation, action and reward design for the dynamic FJSP setting. Using a centralised training phase, each agent (i.e., robot) in the assembly cell executes decentralised scheduling decisions based on local observations. Our solution demonstrated improved performance against rule-based heuristic methods, for optimising makespan. We also reported the impact of different observation sizes of each agent on optimisation performance.**

***Index Terms*—Reinforcement learning, intelligent and flexible manufacturing, double DQN, flexible job shop scheduling problem (FJSP), multi-robot systems, mass personalisation.**

I. Introduction

IN THE rapid evolution to mass personalisation [1], factories of the future are striving for ultra-flexible production of highly personalised products at dynamic batch sizes with mass efficiency. In mass personalisation, jobs with heterogeneous requirements arrive dynamically over time, requiring scheduling decisions to be made on the fly [2]. Traditional meta-heuristic algorithms [3]–[6] for Flexible Job Shop Scheduling Problems (FJSP) obtain near optimal manufacturing schedules under static job instances and environments [7]. They cannot effectively handle any new job arrivals.

Manuscript received 28 February 2022; accepted 14 June 2022. Date of publication 20 June 2022; date of current version 29 June 2022. This letter was recommended for publication by Associate Editor T. Osa and Editor J. Kober upon evaluation of the reviewers' comments. *(Corresponding author: Yuqian Lu.)*

Dazzle Johnson and Yuqian Lu are with the Department of Mechanical and Mechatronics Engineering, The University of Auckland, Auckland 1010, New Zealand (e-mail: djoh596@aucklanduni.ac.nz; yuqian.lu@auckland.ac.nz).

Gang Chen is with the School of Engineering and Computer Science, Victoria University of Wellington, Wellington 6011, New Zealand (e-mail: aaron.chen@ecs.vuw.ac.nz).

Digital Object Identifier 10.1109/LRA.2022.3184795

To cope with the challenges imposed by dynamically arriving jobs, we explore Reinforcement Learning (RL) to address the ultra-flexible manufacturing control problem in mass personalisation.

Prior RL approaches for FJSP rely on single-agent centralised systems, despite the presence of multiple machines, and often only considered fixed job arrival patterns [8]–[11]. Centralised systems require the aggregation of system knowledge in one central machine, increasing communication overhead and vulnerability to failures. Multi-Agent Reinforcement Learning (MARL), under a decentralised architecture, can speed up computation time with parallel computation; allow for the exchange of information using inter-robot communication; and remove the single point of failure caused by the breakdown of a central scheduling machine [12]. As far as we are aware, research on MARL for dynamic FJSP is so far limited. One recent work used a hierarchical multi-agent approach for the dynamic FJSP setting [13]. However, the hierarchical model meant that decision-making is still centrally carried out. A local independent decision-making model would improve flexibility to keep up with the trends of smart manufacturing.

In view of the above, we propose a Multi-Agent Reinforcement Learning system to coordinate a team of collaborative and self-adaptive robots (robots that adapt to and respond to a changing environment) to minimise the makespan in completing a collection of heterogeneous assembly jobs arriving dynamically over time. This paper makes the following contributions through developing and evaluating this MARL system:

1) We design and develop a new scheduling system, with a centralised training phase, that allows for the execution of decentralised scheduling policies (i.e. CTDE paradigm) with local independent decision-making and limited inter-agent communication;
2) We develop the first MARL approach for the dynamic FJSP with dynamic job arrival processes for different fixed team configurations;
3) We extensively evaluate the feasibility and effectiveness of our proposed approach through comparison against rule-based heuristic methods.

The rest of this paper is organised as follows. Section II presents the work done in this area. Section III outlines the problem formulation followed by the algorithm design in Section IV. Experiment results are shown in Section V, benchmarked against

2377-3766 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 08,2022 at 20:26:59 UTC from IEEE Xplore. Restrictions apply.

rule-based heuristic methods. Concluding remarks are presented in Section VI.

II. Related Works

Future manufacturing demands ultra-flexible smart manufacturing systems that can self-adapt to production process changes resulting from varied batch sizes of personalised products [1]. In this context, manufacturing control needs to evolve into self-organised collaboration among a network of autonomous manufacturing things that can adaptively self-optimise against dynamic job arrival patterns and a changing system environment [2].

These new requirements pose a challenge to the traditional meta-heuristic FJSP algorithms [3]–[5] that rely on static job instances and environments [7]. They assume all the jobs and manufacturing factors, such as cost, machine availability, and transportation time, are known and fixed prior to operation. However, these assumptions do not stand in reality and are impossible to be satisfied in mass personalisation [1], [2].

To achieve self-adaptive manufacturing control towards mass personalisation, RL emerged as a promising technology for scheduling [10], [11], [14], [15], due to its ability to learn from past experiences. For allocating a set of tasks to a team of homogeneous robots, [10], [11] developed a solution based on a graph attention network that allows for their solution to be scaled to larger unseen problems. However, these works assume a static environment with static job instances. [14] extends the standard RL Deep Q-Network (DQN) method to a multi-class DQN method for solving the static Job Shop Scheduling Problem (JSP), which assumes each operation only has one feasible machine. [15] on the other hand uses a shared DQN approach to solve the static FJSP setting where all jobs are assumed to be waiting at the start of scheduling.

Some recent RL works have considered certain dynamic settings in their scheduling problem. [16] for example used a DQN network to achieve a mixed rule policy for the control of AGVs. Each AGV is considered capable of completing all tasks, therefore being a much simpler problem than FJSP. A recent work considered resource preemption in a Flexible Flow Shop Scheduling Problem (FFSSP) [17]. They use a MARL approach known as QMIX. Although, their solution is effective in coordinating multiple robots, all jobs are considered to be homogeneous, having the same operation sequence. This assumption cannot hold in an ultra-flexible smart manufacturing environment.

The more challenging dynamic FJSP setting also has been attempted using DQN for selecting dispatching rules [18]. Although, the work in [18] considered transportation times in their FJSP formulation, which were often ignored by conventional static FJSP methods, the arrival of jobs are still static and required to be known a priori. [19] considered dynamic job arrivals for a FJSP setting by responding to new job insertions using a Double DQN architecture. However, these works are based on a single-agent centralised model and full observability of the environment is assumed.

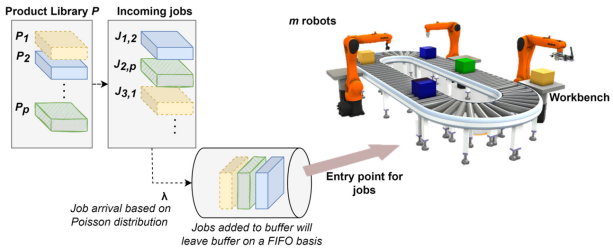


Fig. 1. Assembly cell environment with incoming jobs, generated from a product library, and m machines where jobs arrive with an arrival rate of λ following a Poisson distribution pattern. Buffer available for when conveyor at capacity. Jobs are added onto the conveyor at the entry point and revolve around the belt. Robots can pick up a desired job and place it on a workbench adjacent to them to begin working.

[13] used a hierarchical multi-agent proximal policy optimisation method (HMAPPO) to address the dynamic FJSP setting. Although a multi-agent system approach is used, the hierarchical approach means that decision-making is still centrally carried out by a central controller.

A true multi-agent decentralised system, where each agent only uses local observations to make decisions has been studied in [20], [21]. These works have not been proven to be effective on the FJSP setting. Additionally, coordination is promoted through communication channels between neighbouring agents. In our work, we want to explore a multi-agent system with limited inter-agent communication to minimise communication costs.

Our review of the literature reveals the necessity and urgency of developing a decentralised multi-agent system with local-decision-making capabilities and limited inter-agent communication for diverse and flexible factory settings. There is a lack of consideration for the dynamic arrival of jobs among the existing MARL approaches for dynamic FJSP. We aim to address this gap with our proposed work.

III. Problem Formulation

We aim to solve a real-time dynamic FJSP production scheduling problem in a robot assembly cell as illustrated in Fig. 1. A robot assembly cell consists of a set of m robots with incoming jobs. We assume that the job arrival pattern follows a Poisson distribution with arrival rate λ, according to queuing theory. For testing and training, the simulation is terminated once n jobs have been completed. M denotes the set of all robots. Each job $J_{j,i}$, where $J_{j,i}$ is the jth job entering the assembly cell and it is of product P_i, is chosen from the product library set P, which consists of p products. $S^k_{i,l}$ is the kth processing step of product P_i and it is of operation type O_l. O is the set of unique operation types that can be performed by the set of M robots and consists of o unique operation types. Each processing step $S^k_{i,l}$ of job $J_{j,i}$ must be processed by one robot M_r out of a set of capable robots M^l, where $M^l \subseteq M$, that can carry out operation type O_l.

The conveyor in the assembly cell is divided into c sections, where each section $C_q \in C$ occupies a fixed position relative to the entry point C_1. Section C_q is adjacent to C_{q+1} for $q < c$ and

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 08,2022 at 20:26:59 UTC from IEEE Xplore. Restrictions apply.

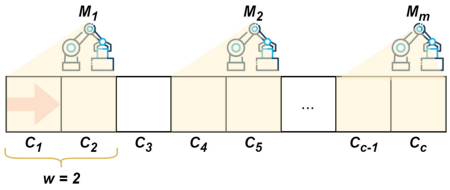


Fig. 2. This illustration shows a flattened view of the conveyor belt divided into c sections. The highlighted sections show the sections viewed by each robot when $w = 2$.

section C_c is adjacent to C_1, creating a loop. Each section can hold at most one job. A flattened conveyor belt is shown in Fig. 2. The percentage that the conveyor is allowed to be filled is defined by c_{max}. Let $\hat{J}_{cell,t}$ be the number of jobs in the assembly cell at timestep t, and let $\hat{J}_{con,t}$ be the number of jobs on the conveyor at timestep t, i.e., $\hat{J}_{con,t}$ excludes jobs in the assembly cell that have been removed from the conveyor by a robot for processing. Jobs are only added at section C_1 of the conveyor if $\hat{J}_{cell,t} < c$, i.e., the number of jobs in the cell should not exceed the number of conveyor sections; $\hat{J}_{con,t} < c \times c_{max}$, i.e., the number of jobs on the conveyor should not exceed the defined fill capacity of the conveyor; and section C_1 at timestep t is empty. If these requirements are not met, the job is added to a buffer and will be added to the conveyor whenever all requirements are met, on a first-in-first-out (FIFO) basis.

The conveyor moves by one section at every timestep t, where the contents of section C_q at timestep t move to section C_{q+1} at timestep $t+1$. The m robots are each positioned adjacent to one section of the conveyor belt, where the position of robot M_r is given by y_r which corresponds to the section that the robot M_r is adjacent to. Robot M_r is able to observe the contents of w sections, i.e., sections $y_r, y_r - 1, \ldots, y_r - w + 1$ are observed, where w is known as the window size. An example is shown in Fig. 2 where $w = 2$. Robot M_1, for example, which is positioned adjacent to section C_2, will observe sections C_2 and C_1 as highlighted, i.e., the robot is able to view the upcoming jobs in the sections that are physically close to the robot. A robot M_r may only retrieve jobs from section y_r. For instance, robot M_1 in Fig. 2 may only retrieve jobs from section C_2.

For operation O_l, every $M_r \in M^l$, operates at a speed based on their speed coefficient $v_{r,l}$. Let $T_{S^k_{i,l}}$ be the base processing time for processing step $S^k_{i,l}$, then the processing time for a given robot M_r to complete processing step $S^k_{i,l}$ is given by $T^r_{S^k_{i,l}}$ as shown in (1).

$$T^r_{S^k_{i,l}} = \frac{T_{S^k_{i,l}}}{v_{r,l}} \tag{1}$$

All notations used to describe the environment are summarised in Table I.

In this paper, the following assumptions are made:

- Processing steps of a job must take place in sequence in the predetermined order for a particular product, i.e., $S^k_{i,l}$ must be completed before $S^{k+1}_{i,l}$;
- For a given product, an operation type O_l can appear in none, one or more processing steps of assembling that product;
- A robot at any timestep can only process one processing step of a single job;
- A robot must complete any processing step of a job once started, i.e., preemption is not allowed;
- A robot can immediately return a job to the closest empty section on the conveyor after completion of a processing step;
- A robot must remove a job from the conveyor to a workbench when working on the job as shown in Fig. 1;
- Completed jobs are not returned to the conveyor and are removed from the assembly cell.

These assumptions are reflective of real manufacturing systems and are consistent with general FJSP settings [19].

TABLE I
TABLE OF NOTATIONS

Symbol	Definition
Indices	
r	Index of robots (agents)
i	Index of product
j	Index of job
k	Index of processing step
l	Index of operation type
q	Index of conveyor section
Constants	
m	Number of robots in assembly cell
n	Number of jobs to be completed before termination
λ	Arrival rate of jobs
p	Number of products
o	Number of unique operation types
c	Number of conveyor sections
c_{max}	Maximum capacity of conveyor expressed as a percentage
w	Window size, defined by the number of conveyor sections a robot can view
Sets	
M	Set of robots, $M = \{M_1, M_2, \ldots, M_m\}$
M^l	Set of capable robots for operation type O_l, where $M^l \subseteq M$
P	Set of products, $P = \{P_1, P_2, \ldots, P_p\}$
O	Set of unique operation types, $O = \{O_1, O_2, \ldots, O_o\}$
O^r	Set of operation types that robot M_r is capable of performing, where $O^r \subseteq O$
C	Set of conveyor sections, $C = \{C_1, C_2, \ldots, C_c\}$
Variables	
$J_{j,i}$	j^{th} job entering assembly cell and it is of product P_i
$S^k_{i,l}$	k^{th} processing step of product P_i and it is of operation type O_l
y_r	Position of robot M_r
$v_{r,l}$	Speed coefficient of robot M_r for operation type O_l
$\hat{J}_{con,t}$	Number of jobs on conveyor at timestep t
$\hat{J}_{cell,t}$	Number of jobs in assembly cell at timestep t, i.e., jobs on conveyor and jobs being processed
$\hat{N}_{done,t}$	Number of processing steps completed at timestep t
$T_{S^k_{i,l}}$	Base processing time for processing step $S^k_{i,l}$
$T^r_{S^k_{i,l}}$	Processing time for processing step $S^k_{i,l}$ for robot M_r
Robot Observation	
O^r_t	Operation type in use by robot M_r at timestep t; $O^r_t = 0$ if no operation type in use
Z_t	Vector of robot statuses for all $M_r \in M$ at timestep t
$\hat{S}_{q,t}$	Remaining processing steps to be completed on a job in section C_q at timestep t

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 08,2022 at 20:26:59 UTC from IEEE Xplore. Restrictions apply.

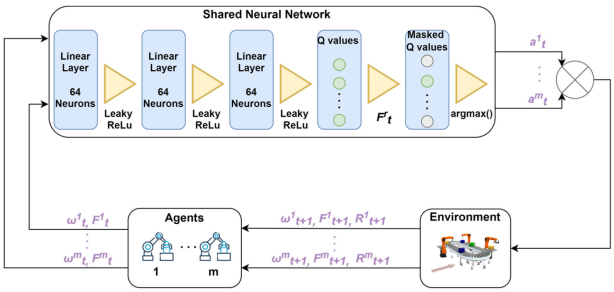


Fig. 3. Multi-Agent Reinforcement Learning system with shared neural network architecture, which takes as input (ω_t^r, F_t^r) and outputs an action a_t^r, with filtering of Q values that correspond to infeasible actions.

In this paper, the optimisation objective is to minimise the makespan, defined by the maximal completion time of a series of jobs.

IV. Algorithm Design

A. Formulation of the Dynamic FJSP as a Multi-Agent Reinforcement Learning Problem

We model the dynamic FJSP introduced in Section III as a MARL system. MARL consists of multiple agents. Each agent can interact with their shared environment by performing an action and receiving feedback from the environment in the form of an observation and reward, as shown in Fig. 3.

Following the multi-agent model, where each robot is modelled as an agent, the MARL problem is defined as $(\mathcal{N}, \mathcal{S}, \mathcal{A}, \mathcal{P}, \{\mathcal{R}^r\}_{r\in\mathcal{N}}, \Omega, \gamma)$ where $\mathcal{N}$ denotes the set of all agents, $\mathcal{S}$ is the state space, $\mathcal{A}$ represents the action space, Ω is the observation space and $\gamma \in [0, 1)$ is the discount factor. At each timestep t, every agent $r \in \mathcal{N}$ receives a partial observation $\omega_t^r \in \Omega$ where $\hat{\omega}_t$ is the joint observation. The agent takes an action $a_t^r \in \mathcal{A}$, and receives a reward $\mathcal{R}_t^r \in \{\mathcal{R}^r\}_{r\in\mathcal{N}}$, as shown in Fig. 3. Applying a joint action $\hat{a}$ in state $\hat{s} \in \mathcal{S}$ results in a transition to state $\hat{s}' \in \mathcal{S}$ with a probability $\mathcal{P}$. All agents work cooperatively to maximise the system's total expected, discounted reward, i.e., maximise $\mathbb{E}[\sum_{t>0} \sum_{r=1}^{r=m} \gamma^t \mathcal{R}_t^r(\omega_t^r, a_t^r)]$.

The observation space, action space and reward function are defined in the following sections.

1) Observation:

$$\omega_t^r = [y_r, O^r, O_t^r, Z_t, \hat{S}_{y_r,t}, \hat{S}_{y_r-1,t}, \ldots, \hat{S}_{y_r-w+1,t}] \tag{2}$$

Each agent makes its own local observations. The observation of each agent at timestep t is defined by ω_t^r in (2). The first element of the observation vector is the agent's position, y_r, followed by O^r. O^r is the set of operation that agent r can carry out, where $O^r \subseteq O$. Since all agents use a shared DQN network, it is important for the network to distinguish the decision-making agent and capture the relationship among agents. Overlapping capabilities and distances between agents are indirectly captured due to the use of a shared neural network. Note that the locations of the robots are fixed and therefore remain the same across training and testing.

TABLE II
Enumeration of Agent Statuses

State	Description
0	Agent is **idle**
1	Agent **accepts** a processing step of a job
2	Agent is **working** on a processing step of a job
3	Agent has **completed** a processing step of a job

The third element, O_t^r, is the current operation type that is performed by agent r at timestep t. $O_t^r = 0$ if no operation type is in use.

The next component of the observation space, Z_t, captures the status of all agents at timestep t. Inter-agent communication is required to determine Z_t. This information exchange can be performed scalably and efficiently. The possible statuses of the agents are defined in Table II.

The final component of the observation space is defined as follows, let $\hat{S}_{q,t}$ be the remaining operations required to be completed on a waiting job in section q of the conveyor belt, then the agent r observes $\hat{S}_{y_r,t}, \hat{S}_{y_r-1,t}, \ldots, \hat{S}_{y_r-w+1,t}$ at timestep t. If there is no job in any of the $y_r, y_r - 1, \ldots, y_r - w + 1$ sections, the observation space is padded with zeros.

2) Action:

$$\mathcal{A} = [A_0, A_1, \ldots, A_{w-1}, A_d, A_c] \tag{3}$$

The action space for each agent r is shown in (3). A_0 corresponds to the **Accept** action where the agent accepts and begins work on a job that is either in section y_r of the conveyor or on the workbench of the agent. If the job is on the conveyor, it will be removed by the agent and placed on the workbench; if it is on the workbench, the next processing step of the job will be started. Actions $A_1 \ldots, A_{w-1}$ correspond to the **Wait** action $\hat{A}_x$, and represents the agent's decision to wait for one of the upcoming jobs in sections $y_r - 1$ to $y_r - w + 1$. The inclusion of a **Wait** action allows agents to receive immediate feedback, in the form of a reward, regarding the benefits of allowing the job directly in front of them to pass by in preference of another upcoming job. This aids learning and guides the agents to collaboratively determine good schedules on the fly. Note that the agent selects an action at every timestep, therefore selecting a **Wait** action still requires selection of the **Accept** action on that job when it arrives at y_r. A_d and A_c are the **Decline** and **Continue** actions respectively. When the **Decline** action is chosen, the agent declines the job in front of it and the agent is not waiting for any upcoming jobs in its view. The **Continue** action is the default choice for an agent that is not required to make a decision about a job, i.e., when there are no jobs in the agents' view or the agent is currently processing a job.

To aid efficient learning, an action filtering technique similar to [22] is applied. Since not all actions are feasible at all times, we create a mask F_t^r for each agent to determine which actions can and cannot be taken by the agent based on the operation type required by the jobs and the operation type capabilities of the agent. The filtering process is applied by feeding the mask as an input to the neural network and the Q values of the corresponding actions are filtered as illustrated in Fig. 3.

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 08,2022 at 20:26:59 UTC from IEEE Xplore. Restrictions apply.

This is done by setting the Q value outputs to negative infinity, meaning the corresponding action will not be chosen during the $\arg\max()$ operation. During training, the mask for future actions is also obtained to filter out invalid future actions.

3) Reward:

$$\alpha = 1.1, \zeta = 1.0, \beta = 1.0, \delta = 0.5 \tag{4}$$

$$\mathcal{R}_t^r = -\alpha + \zeta R_{step}(t) + \beta R_{proc}^r(t) + \delta R_{wait}^r(t) \tag{5}$$

The reward for each agent r at timestep t is shown in (5) with coefficients shown in (4). We define the reward function to achieve the ultimate goal of minimising the makespan. For this purpose, a penalty of α is given to each agent at every timestep, encouraging a faster completion time.

$$R_{proc}^r(t) = \begin{cases} \frac{v_{r,l}}{\sum_{i=1}^{|M^l|} v_{i,l}}, & \text{if } Z_t^r = 2 \\ 0, & \text{otherwise.} \end{cases} \tag{6}$$

$$R_{wait}^r(t) = \begin{cases} \frac{v_{r,l}}{x \sum_{i=1}^{|M^l|} v_{i,l}}, & \text{if } \textbf{Wait} \text{ action taken} \\ 0, & \text{otherwise.} \end{cases} \tag{7}$$

Through R_{proc} in (5), individual agents are rewarded for working. Furthermore, $R_{wait}^r(t)$ rewards waiting and allowing a job to pass by to reduce the overall makespan.

Reward $R_{proc}^r(t)$ and $R_{wait}^r(t)$, summarised in (6) and (7) respectively, are both proportional to $v_{r,l}$, i.e., if the agent is faster than the other agents for a particular operation type, it will receive a higher reward. Given that a **Wait** action $\hat{A}_x \in \{A_1, A_2, \ldots, A_{w-1}\}$ is executed by agent r, where x represents the number of sections between the agent and job, the reward is scaled down by x, taking transportation time into account and increasing focus on nearby jobs. The **Wait** action is rewarded to guide the agents' decision making towards long-term returns by encouraging waiting for more favourable jobs.

$$R_{step}(t) = \hat{N}_{done,t} \tag{8}$$

In addition, all agents receive an equal reward $R_{step}(t)$ according to (8), where $\hat{N}_{done,t}$ is the total number of processing steps completed at timestep t. This shared reward encourages cooperation among the agents.

Each term in the reward function is essential for agents to collectively achieve an improved average makespan and aid faster learning.

Selection of coefficients α, β, δ and ζ follow a trial-and-error approach to find an appropriate composite reward to guide agents to minimise makespan. The chosen values in our experiments are summarised in (4).

B. Algorithm Architecture

The Double DQN algorithm was applied to our problem, as shown in Algorithm 1. The Double DQN architecture mitigates the overestimation problem of a standard DQN architecture [23]. Experience replay with a target network was used to aid stability. The neural network architecture, which is commonly used in literature, is shown in Fig. 3, which consists of three hidden layers using a Leaky ReLu activation function to minimise the gradient exploding challenges.

Algorithm 1: Double DQN-based Algorithm for Dynamic Multi-Agent Robot Assembly Scheduling.

Initialise shared network $\mathcal{Q}_\theta$;
Initialise shared target network $\mathcal{Q}'_\theta$;
Initialise replay buffer $\mathcal{D}$;
$\tau << 1$;
$\epsilon = 1$;
Initialise factory environment $\mathcal{F}$;
for *each episode* **do**
  Reset $\mathcal{F}$;
  Get job instance and arrival time for each job;
  for *each episode step* **do**
    for *each agent* r **do**
      Make observation ω_t^r;
      Get mask F_t^r;
      With probability ϵ select random action a_t^r from set of feasible actions $\mathcal{A}'$;
      Else select action a_t^r = argmax$_{a^r}$ $\mathcal{Q}_\theta(\omega_t^r, F_{r,t}, a^r)$;
      Store selected action a_t^rin $\hat{a}_t$;
    end
    Execute joint action $\hat{a}_t$;
    Make new observations ω_{t+1}^r, masks F_{t+1}^r and rewards $\mathcal{R}_t^r$ for each agent r;
    Store $(\omega_t^r, F_{r,t}, a_t^r, (R)_t^r, \omega_{t+1}^r, F_{r,t+1})$ for each agent r in replay buffer $\mathcal{D}$ if $\hat{\omega}_t \neq \hat{\omega}_{t+1}$;
    for *each update step* **do**
      sample $e_t = (\omega_t^r, F_{r,t}, a_t^r, (R)_t^r, \omega_{t+1}^r, F_{r,t+1}) \sim \mathcal{D}$ for each agent r;
      Compute $\mathcal{Q}^*(\omega_t^r, F_{r,t}, a_t^r) = (R)_t^r + \gamma \mathcal{Q}_\theta(\omega_t^r, F_{r,t}, argmax_{a^{r'}} \mathcal{Q}'_\theta(\omega_{t+1}^r, F_{r,t+1}, a^{r'}))$;
      Perform gradient descent step on $(\mathcal{Q}^*(\omega_t^r, F_{r,t}, a_t^r) - \mathcal{Q}_\theta(\omega_t^r, F_{r,t}, a_t^r))^2$;
      Update target network parameters: $\theta' \leftarrow \tau \times \theta + (1 - \tau) \times \theta'$;
    end
  end
  Decay ϵ
end

The overall algorithm and system functionality is shown in Algorithm 1 and Fig. 3. Each agent makes an observation and an action will be chosen as per the Double DQN-based algorithm. Each agent will concurrently carry out the selected actions, make new observations, and receive rewards. Each transition will be stored in the buffer if the next state differs from the current state. Samples will be taken from the buffer to update and train the network. The episode terminates when n jobs have been completed, or the maximum number of episode steps have been reached.

The experience replay is used to retrieve concurrent samples as designed in [24], i.e., the states, actions and rewards of all agents at a given timestep index t are used to train the neural

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 08,2022 at 20:26:59 UTC from IEEE Xplore. Restrictions apply.

TABLE III
ENVIRONMENT PARAMETER SETTINGS

	3 Agents	**6 Agents**	**12 Agents**
m	3	6	12
n	20	20	40
c	12	12	24
c_{max}	0.75	0.75	0.75
O^r	$O^1 = 1, 2,$ $O^2 = 2, 3,$ $O^3 = 1, 3$	$O^1, O^4 = 1, 2,$ $O^2, O^5 = 2, 3,$ $O^3, O^6 = 1, 3$	$O^1, O^4, O^7, O^{10} = 1, 2,$ $O^2, O^5, O^8, O^{11} = 2, 3,$ $O^3, O^6, O^9, O^{12} = 1, 3$

TABLE IV
ARRIVAL RATES TESTED FOR EACH TEAM SIZE

Busyness	3 Agents	6 Agents	12 Agents
Low	0.015	0.030	0.060
Medium	0.020	0.040	0.080
High	0.030	0.060	0.120

TABLE V
TRAINING PARAMETERS

Parameter	Value
Episodes	2000
Max Episode Steps	2000
Batch Size	64
Buffer Size	3×10^5
τ	1×10^{-4}
$\epsilon_{initial}$	1
ϵ_{final}	0.1
Learning Rate	1×10^{-4}
γ	0.98
Learning Frequency	2
Number of experiences to be stored prior to training	15×10^5

network. This allows for interactions between agents to be considered and improves performance [24].

We introduced a more efficient use of the replay buffer. For state transitions where the system's adjacent states remain unchanged, i.e., the next state is equivalent to the system's preceding state, state transitions will be ignored and not stored in the buffer. When this occurs, no action selection is required by any agent because all agents are either working or are all idle with no jobs to be observed on the conveyor. This avoids the buffer being largely composed of such experiences and overshadowing more valuable experiences, thus speeding up the training process.

V. EXPERIMENTS AND DISCUSSION

To evaluate the performance of our newly developed MARL system, we simulated various FJSP scenarios in a robot assembly cell using the parameters shown in Table III. These parameters, including the robot positions, remain fixed between training and evaluation. The training parameters are summarised in Table V. For the three team configurations shown in III, we conducted experiments on the arrival rates shown in Table IV. The arrival rates represent low, medium and high levels of busyness for the different team sizes. For each team and arrival rate configuration, we explored the impact of the window size parameter w under different settings of 1, 2, 3 and 4 on makespan.

A. Training and Testing

During simulation, at every episode, jobs are initially added onto the conveyor belt at every timestep until the maximum number of allowable jobs on the conveyor belt has been reached. This is the warm-up period; after this point, jobs enter the system according to a Poisson distribution with an arrival rate of λ. The warm-up period avoids prolonged agent idleness while jobs accumulate on the conveyor.

Jobs are randomly generated from a fixed library of products, where each product has an equal probability of being selected. A different seed is used to generate different job instances and arrival times at every episode.

During the final 1000 episodes of training, every 10th episode is used for testing. During testing, training is paused and the trained model is used for action selection. Therefore, data from 100 testing episodes were collected and used to report results. 10 trials were completed for each parameter setting, using a fixed sequence of seeds to ensure the job instances and the arrival time of jobs were the same across runs and when compared to the benchmarking methods. Thus, 10 different DQN networks under each setting were trained and the performance of each is averaged to obtain the final results.

B. Competing Algorithms

We use makespan as a performance indicator, and our solution is benchmarked against well-known dispatching rules used in previous studies [13], [16], [18]. We do not compare our MARL system with any meta-heuristic and RL-based approaches. In fact, according to our literature review in Section II, all existing techniques cannot be directly applied to our modelled dynamic FJSP setting. The following rules were used to decide whether an agent accepts a job to process:

1) *RAND:* Random selection of actions
2) *MWR:* Most work remaining of jobs on conveyor, i.e., job with the most remaining processing steps
3) *LPT:* Longest average processing time for job on conveyor
4) *SPT:* Shortest average processing time for job on conveyor
5) *FCFS:* First come first serve jobs
6) *FAA:* Fastest available agent for job accepts job

C. Analysis

The average makespan achieved by the proposed MARL solution for $m = 3, 6$ and 12 with varying window sizes and arrival rates is shown in Figs. 4–6 respectively, benchmarked against the heuristic methods. Table VI shows the p-values obtained from the Wilcoxon analysis for a pair-wise comparison between the proposed MARL method and heuristic methods. Table VII shows the p-values obtained from the Wilcoxon analysis to verify whether there is a statistically significant difference among different window sizes. Statistically significant values are bolded.

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 08,2022 at 20:26:59 UTC from IEEE Xplore. Restrictions apply.

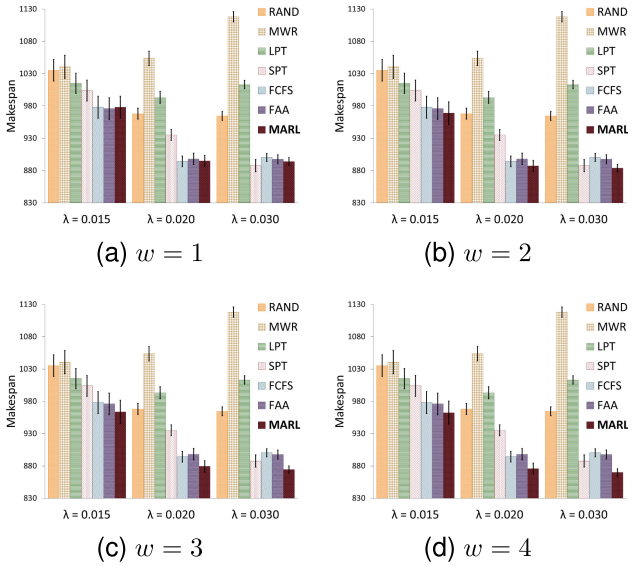


Fig. 4. Average makespan achieved using the proposed solution at arrival rates of $\lambda = 0.015$, $\lambda = 0.020$ and $\lambda = 0.030$ for $m = 3$ and $n = 20$, benchmarked against heuristic rules. (a)–(d) shows the results at $w = 1$ to $w = 4$ respectively. The result of MARL is the average performance of 10 trained DQN networks.

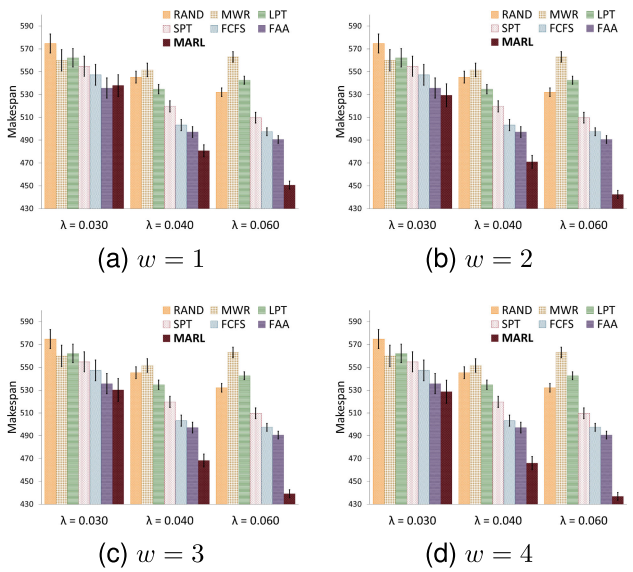


Fig. 5. Average makespan achieved using the proposed solution at arrival rates of $\lambda = 0.030$, $\lambda = 0.040$ and $\lambda = 0.060$ for $m = 6$ and $n = 20$, benchmarked against heuristic rules. (a)–(d) shows the results at $w = 1$ to $w = 4$ respectively. The result of MARL is the average performance of 10 trained DQN networks.

TABLE VI
P-VALUES OBTAINED USING WILCOXON SIGNED-RANK TEST COMPARING METHODS AT A SIGNIFICANCE LEVEL OF 0.05

		$w=1$			$w=2$			$w=3$			$w=4$		
λ		$m=3$	$m=6$	$m=12$	$m=3$	$m=6$	$m=12$	$m=3$	$m=6$	$m=12$	$m=3$	$m=6$	$m=12$
Low	RAND	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5
	MWR	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5
	LPT	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5
	SPT	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5
	FCFS	==	<1E-5	7.64E-1	**5.80E-4**	<1E-5	2.54E-1	<1E-5	<1E-5	9.69E-2	<1E-5	<1E-5	2.38E-1
	FAA	4.30E-1	2.01E-1	7.64E-1	**4.14E-2**	4.77E-2	3.90E-1	**5.00E-4**	1.31E-1	1.53E-1	**1.60E-4**	**3.49E-2**	2.08E-1
Med	RAND	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5
	MWR	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5
	LPT	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5
	SPT	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5
	FCFS	8.73E-1	<1E-5	<1E-5	8.91E-2	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5
	FAA	5.49E-1	<1E-5	6.15E-2	**1.78E-2**	<1E-5	**2.20E-2**	<1E-5	<1E-5	**2.20E-2**	<1E-5	<1E-5	**7.60E-4**
High	RAND	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5
	MWR	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5
	LPT	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5
	SPT	4.78E-1	<1E-5	<1E-5	6.53E-1	<1E-5	<1E-5	5.24E-2	<1E-5	<1E-5	**9.06E-3**	<1E-5	<1E-5
	FCFS	1.12E-1	<1E-5	<1E-5	**9.40E-4**	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5	<1E-5
	FAA	6.17E-1	<1E-5	2.08E-1	**8.60E-4**	<1E-5	2.19E-1	<1E-5	<1E-5	**2.62E-3**	<1E-5	<1E-5	<1E-5

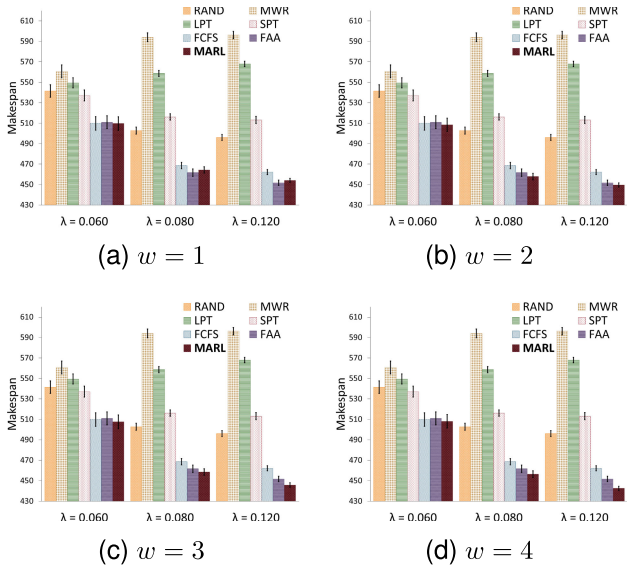


Fig. 6. Average makespan achieved using the proposed solution at arrival rates of $\lambda = 0.060$, $\lambda = 0.080$ and $\lambda = 0.120$ for $m = 12$ and $n = 40$, benchmarked against heuristic rules. (a)–(d) shows the results at $w = 1$ to $w = 4$ respectively. The result of MARL is the average performance of 10 trained DQN networks.

TABLE VII
P-VALUES OBTAINED USING WILCOXON SIGNED-RANK TEST COMPARING WINDOW SIZES AT A SIGNIFICANCE LEVEL OF 0.05

		Low λ				Medium λ				High λ			
m		$w=1$	$w=2$	$w=3$	$w=4$	$w=1$	$w=2$	$w=3$	$w=4$	$w=1$	$w=2$	$w=3$	$w=4$
3	$w=1$		**5.80E-4**	<1E-5	<1E-5		**7.36E-3**	<1E-5	<1E-5		**1.24E-2**	<1E-5	<1E-5
	$w=2$			<1E-5	<1E-5			**1.40E-4**	<1E-5			<1E-5	<1E-5
	$w=3$				2.26E-1				**3.57E-2**				**1.35E-2**
6	$w=1$		<1E-5	<1E-5	<1E-5		<1E-5	<1E-5	<1E-5		<1E-5	<1E-5	<1E-5
	$w=2$			3.84E-1	4.41E-1			**7.36E-3**	<1E-5			**1.32E-3**	<1E-5
	$w=3$				5.74E-2				**3.16E-2**				**1.93E-2**
12	$w=1$		2.50E-1	8.19E-2	2.08E-1		<1E-5	<1E-5	<1E-5		<1E-5	<1E-5	<1E-5
	$w=2$			1.21E-1	5.42E-1			5.16E-1	**1.64E-2**			<1E-5	<1E-5
	$w=3$				3.52E-1				**1.64E-2**				<1E-5

The results show that our proposed method achieves an improved performance with higher arrival rates and larger window sizes across all settings as shown in Figs. 4–6. The statistical analysis shown in Table VI where each heuristic method is compared against the proposed method, show that for a window size of 4 and medium and high arrival rates, there is a statistically significant improvement in performance. For a low arrival rate, the performance is comparable to that of FCFS and FAA heuristic methods. As the arrival rate increases, the improvement in makespan of our proposed method also increases. For example, in Fig. 5(d) under $m = 6$ and $w = 4$, the performance improvement compared to FAA is 1.3%, 6.3% and 11.0% for $\lambda = 0.03, 0.04$ and 0.06 respectively. This trend in performance improvement with an increased arrival rate is observed across all three settings. Table VII shows the statistical analysis conducted to compare the window sizes under each setting. The results show that, under higher arrival rates, there is a statistically significant difference among all window sizes. Performance also improves noticeably upon enlarging the window sizes. Under a low arrival rate there are cases where there is no significant difference among the window sizes. For example, under the lowest arrival rate, a window size of 3 and 4 were not statistically different in performance under each setting. A difference in performance can be explained by the agents

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 08,2022 at 20:26:59 UTC from IEEE Xplore. Restrictions apply.

being able to view more jobs within their windows, enabling better decision-making. Whereas, a similar performance across window sizes may indicate a similar number of jobs being viewed within a window at a given timestep. Therefore, careful selection of the window size based on the arrival rate is required.

VI. Conclusion

This paper proposes a Multi-Agent Reinforcement Learning solution based on Double DQN for a dynamic FJSP setting in a robot assembly cell. We introduced a scheduling system, with a centralised training phase, that allows for the execution of decentralised scheduling policies. Only local observations were made by agents and the communication between agents was limited to minimise communication costs compared to existing MARL methods. The proposed solution can handle dynamic job arrival processes under a MARL system. The proposed method showed excellent performance compared to the six heuristic rules for higher arrival rates and larger window sizes. Using the proposed solution, shorter makespans were achieved consistently, improving the overall efficiency of the robot assembly cell. The algorithm was validated on a conveyor case study, however the design of the MARL system is generally applicable to the commonly studied FJSP settings.

Although, this work focused on minimising makespan, other objectives, such as tardiness and machine utilisation, can be considered with a new design of the reward function, adopting our design principles. In order to cope with a wide range of different environment settings for training and testing, one possible future research is to consider whether knowledge can be transferred across environments and generalised to different settings. Further, there is room to evaluate our model performance with other MARL architectures such as QMIX with an adapted neural network architecture.

Acknowledgment

We thank Weiliang Xu for providing his comments. We also thank the Editors and Reviewers for their valuable comments and suggestions in improving the quality of this paper.

References

[1] Y. Lu, X. Xu, and L. Wang, "Smart manufacturing process and system automation – A critical review of the standards and envisioned scenarios," *J. Manuf. Syst.*, vol. 56, pp. 312–325, Jul. 2020.

[2] Z. Qin and Y. Lu, "Self-organizing manufacturing network: A paradigm towards smart manufacturing in mass personalization," *J. Manuf. Syst.*, vol. 60, pp. 35–47, Jul. 2021.

[3] J. Liangxiao and D. Zhongjun, "An improved genetic algorithm for flexible job shop scheduling problem," in *Proc. 2nd Int. Conf. Inf. Sci. Control Eng.*, 2015, pp. 127–131.

[4] G. Zhang, L. Gao, and Y. Shi, "An effective genetic algorithm for the flexible job-shop scheduling problem," *Expert Syst. Appl.*, vol. 38, no. 4, pp. 3563–3573, 2011.

[5] F. Pezzella, G. Morganti, and G. Ciaschetti, "A genetic algorithm for the flexible job-shop scheduling problem," *Comput. Operations Res.*, vol. 35, no. 10, pp. 3202–3212, 2008.

[6] H.-W. Ge, L. Sun, Y.-C. Liang, and F. Qian, "An effective PSO and AIS-based hybrid intelligent algorithm for job-shop scheduling," *IEEE Trans. Syst., Man, Cybern. - Part A: Syst. Hum.*, vol. 38, no. 2, pp. 358–368, Mar. 2008.

[7] S. Zhang, F. Tang, X. Li, J. Liu, and B. Zhang, "A hybrid multi-objective approach for real-time flexible production scheduling and rescheduling under dynamic environment in industry 4.0 context," *Comput. Operations Res.*, vol. 132, 2021, Art. no. 105267.

[8] J. Zhu, H. Wang, and T. Zhang, "A deep reinforcement learning approach to the flexible flowshop scheduling problem with makespan minimization," in *Proc. IEEE 9th Data Driven Control Learn. Syst. Conf.*, 2020, pp. 1220–1225.

[9] Y. R. Shiue, K. C. Lee, and C. T. Su, "A reinforcement learning approach to dynamic scheduling in a product-mix flexibility environment," *IEEE Access*, vol. 8, pp. 106542–106553, 2020.

[10] Z. Wang and M. Gombolay, "Learning scheduling policies for multi-robot coordination with graph attention networks," *IEEE Robot. Automat. Lett.*, vol. 5, no. 3, pp. 4509–4516, Jul. 2020.

[11] Z. Wang, C. Liu, and M. Gombolay, "Heterogeneous graph attention networks for scalable multi-robot scheduling with temporospatial constraints," *Auton. Robots*, vol. 46, no. 1, pp. 249–268, 2022.

[12] L. Buşoniu, R. Babuška, and B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Trans. Syst., Man Cybern. Part C: Appl. Rev.*, vol. 38, no. 2, pp. 156–172, Mar. 2008.

[13] S. Luo, L. Zhang, and Y. Fan, "Real-time scheduling for dynamic partial-no-wait multiobjective flexible job shop by deep reinforcement learning," *IEEE Trans. Autom. Sci. Eng.*, 2021, doi: 10.1109/TASE.2021.3104716.

[14] C. C. Lin, D. J. Deng, Y. L. Chih, and H. T. Chiu, "Smart manufacturing scheduling with edge computing using multiclass deep Q network," *IEEE Trans. Ind. Inform.*, vol. 15, no. 7, pp. 4276–4284, Jul. 2019.

[15] I. B. Park, J. Huh, J. Kim, and J. Park, "A reinforcement learning approach to robust scheduling of semiconductor manufacturing facilities," *IEEE Trans. Automat. Sci. Eng.*, vol. 17, no. 3, pp. 1420–1431, Jul. 2020.

[16] H. Hu, X. Jia, Q. He, S. Fu, and K. Liu, "Deep reinforcement learning based AGVs real-time scheduling with mixed rule for flexible shop floor in industry 4.0," *Comput. Ind. Eng.*, vol. 149, 2020, Art. no. 106749.

[17] X. Wang, L. Zhang, T. Lin, C. Zhao, K. Wang, and Z. Chen, "Solving job scheduling problems in a resource preemption environment with multi-agent reinforcement learning," *Robot. Comput.-Integr. Manuf.*, vol. 77, 2022, Art. no. 102324.

[18] J. Yan, Z. Liu, T. Zhang, and Y. Zhang, "Autonomous decision-making method of transportation process for flexible job shop scheduling problem based on reinforcement learning," in *Proc. Int. Conf. Mach. Learn. Intell. Syst. Eng.*, 2021, pp. 234–238.

[19] S. Luo, "Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning," *Appl. Soft Comput. J.*, vol. 91, 2020, Art. no. 106208.

[20] K. Zhang, Z. Yang, H. Liu, T. Zhang, and T. Başar, "Fully decentralized multi-agent reinforcement learning with networked agents," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, vol. 80, pp. 5872–5881.

[21] S. Konan, E. Seraj, and M. Gombolay, "Iterated reasoning with mutual information in cooperative and byzantine decentralized teaming," *arXiv2201.08484*, 2022.

[22] M. Seurin, P. Preux, and O. Pietquin, ""I'm sorry dave, I'm afraid I can't do that" deep Q-learning from forbidden actions," in *Proc. Int. Joint Conf. Neural Netw.*, 2020, pp. 1–8.

[23] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 30th AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.

[24] S. Omidshafiei, J. Pazis, C. Amato, J. P. How, and J. Vian, "Deep decentralized multi-task multi-agent reinforcement learning under partial observability," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, vol. 70, pp. 2681–2690.

Authorized licensed use limited to: ULAKBIM UASL - CUKUROVA UNIVERSITESI. Downloaded on November 08,2022 at 20:26:59 UTC from IEEE Xplore. Restrictions apply.